Arrays in Java

Easy notes from the Data Structures and Algorithms lecture: basics, how an array works inside memory, and how to find the address of any element.

DSA series, Week 2

The lecture is split into three parts. These notes cover the first two, which are the ones the whole video teaches, and point to the third at the end.

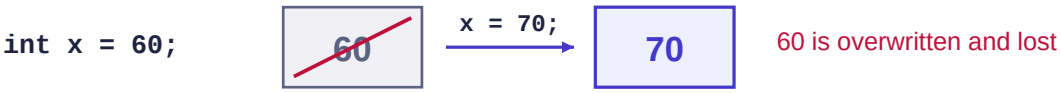
1. **Array basics.** Easy to follow, but the concepts matter a lot.
2. **Internal working.** How memory is allocated, and how to calculate the address of any element.
3. **Array ADT** (advanced). Building an Array Abstract Data Type with all its operations. This comes in the next video. If ADT is new to you, watch the previous video of the series first.

1. What is an array?

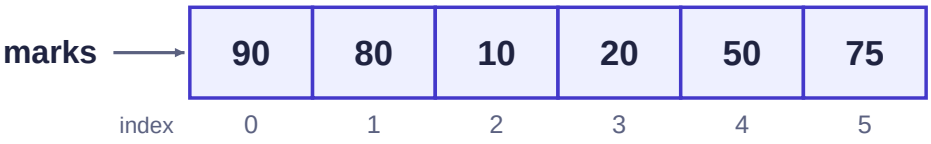
An array lets you keep many values under one variable name. Two points and one picture are all you need.

Point 1: many values in one variable

A normal variable holds one value at a time. If you write `int x = 60;` and later `x = 70;`, the old value 60 is overwritten.



Now imagine storing the names of 10 students, or the marks of 100 students. A hundred separate variables would be painful to create and even harder to use. An array solves this: one variable, for example `marks`, holds all the values.



The variable marks points to the first element. From there, every other element can be reached.

Point 2: same type, side by side

An array is a collection of elements of the **same type**, stored in **contiguous** (side by side) memory locations.

- **Same type** means all int, or all double, or all String, or all objects of one class (all Dog objects, all Student objects).
- **Contiguous** means one after another with no gaps. Think of the rooms of a house: room 1, then room 2 touching it, then room 3 touching that.

> **Key idea**
> Whenever you think of an array, picture the row of boxes above. Everything in this lecture is about that row.

In Python, a list can hold mixed types because Python is not statically typed. In statically typed languages such as Java, C and C++, an array holds one type.

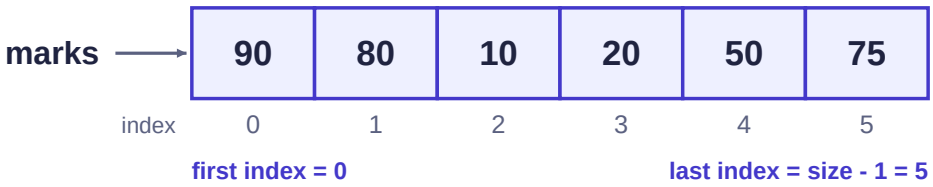
2. Four key characteristics

Characteristic	What it means
Fixed size	The size is given when the array is created and cannot change while the program runs. You can change the values inside, but not how many boxes there are.
Index based	Every element has an index, and the index always starts from 0. You reach an element through its index.
Homogeneous	All elements have the same type.
Contiguous memory	All elements sit next to each other in memory. This is what makes arrays fast (see section 6).

What is size?

Size is the number of elements in the array. The array below has 6 elements, so its size is 6. It was fixed at the moment the array was created. You can later replace 90 with 40 or 80 with 75, but you cannot make the array bigger or smaller.

Index rules



- The first element is always at index **0**.
- The last element is always at index **size - 1**. A size of 6 gives indexes 0 to 5. A size of 100 gives 0 to 99. A size of 200 gives 0 to 199.
- Elements are always stored at an index.

> **Remember**
> To visit every element, run a loop from **0** up to **size - 1**. Many beginners get this limit wrong.

3. Types of arrays

Arrays can have one or more dimensions. 1D and 2D arrays are the ones you will meet most often. If you understand these two, you can understand the rest.

Type	Dimensions	Brackets when declaring	Example
1D array	1	one pair	`int[] marks;`
2D array	2	two pairs	`int[][] table;`
3D array	3	three pairs	`int[][][] cube;`
nD array	n	n pairs	and so on

A 1D array is a single line of data. This lecture works with 1D arrays. 2D arrays come in a later video.

4. Working with arrays in Java

Java gives you arrays ready-made, so you do not have to build one yourself. You only need to know how to use it.

4.1 Declare an array

Write the type, then the variable name, then square brackets.

```
int[] marks;     // an array of int named marks
```

- The square brackets tell Java that this is an array. Forget them and it becomes a normal variable.
- One pair of brackets per dimension.
- Declaring only creates the name. No values are stored yet.

4.2 Give it values

Situation A: you already know the values. Declare and initialize in the same line, using curly braces.

```
int[] marks = {90, 80, 10, 20, 50, 75};
```

This creates marks with 6 elements. The drawback is that you must know every value in advance.

> **Common mistake**
>
> Declaring on one line and using curly braces on a later line is a compile error ("array constants can only be used in initializers"). Do both in one statement.
>
> ```
> int[] marks;
> marks = {90, 80, 10, 20, 50, 75};   // ERROR
> ```
>
> Extra tip, not from the lecture: **`marks = new int[] {90, 80, 10, 20, 50, 75};`** also works on a later line.

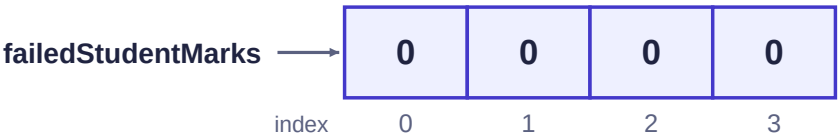
Situation B: you do not know the values yet. For example, the marks of failed students will arrive later. Create the array with a size and fill it afterwards. You can do it in two steps:

```
int[] failedStudentMarks;                // step 1: declare
failedStudentMarks = new int[4];         // step 2: create 4 slots
```

or in one line, with the size kept in a variable:

```
int size = 4;
int[] failedStudentMarks = new int[size];
```

Java fills every slot with the default value of the type:



Array type	Default value in every slot
int	0
double	0.0
boolean	false
String and other objects	null

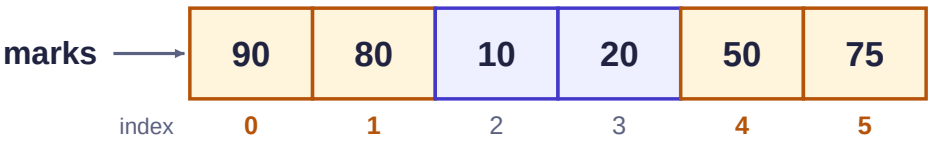
```
System.out.println(failedStudentMarks[0]);
System.out.println(failedStudentMarks[1]);
```

Output

```
0
0
```

4.3 Access an element

Write the array name and the index in square brackets.




```
int[] marks = {90, 80, 10, 20, 50, 75};
System.out.println(marks[0]);   // first element
System.out.println(marks[1]);
System.out.println(marks[4]);
System.out.println(marks[5]);   // last element, index = size - 1
```

Output

```
90
80
50
75
```

Find the size with length

Do not count by hand. Every array has a **`length`** property that gives its size, so the last element is at **`marks.length - 1`**.

```
System.out.println(marks[marks.length - 1]);
```

Output

```
75
```

> **Remember**
>
> **`length`** is a **property**, not a method. Write **`marks.length`** with no brackets. **`marks.length()`** is an error.

4.4 Change an element

Assign to a position the same way you would assign to a normal variable.

```
marks[0] = 100;     // 90 is replaced by 100
```

From now on, reading **`marks[0]`** gives 100.

4.5 Visit every element (traversal)

Going through the elements one by one is called **traversing**. You need it whenever you want to print all values, add them up, search, and so on. Elements are only useful if you do something with them. Both examples below use **`marks = {90, 80, 10, 20, 50, 75}`**.

Method 1: normal for loop. The variable **`i`** is the index. It starts at 0 and runs while **`i < marks.length`**, so its last value is size - 1.

```
for (int i = 0; i < marks.length; i++) {
    System.out.println("Value at index " + i + " is " + marks[i]);
}
```

Output

```
Value at index 0 is 90
Value at index 1 is 80
Value at index 2 is 10
Value at index 3 is 20
Value at index 4 is 50
Value at index 5 is 75
```

Method 2: for-each (enhanced for) loop. No index to manage. Each pass gives you the next value.

```
for (int m : marks) {
    System.out.println(m);
}
```

Output

```
90
80
10
20
50
75
```

Use the normal loop when you need the index. Use for-each when you only need the values.

A complete program

```
public class ArrayExample {
    public static void main(String[] args) {
        int[] marks = {90, 80, 10, 20, 50, 75};
        int[] failedStudentMarks = new int[4];

        System.out.println(marks[0]);                   // 90
        System.out.println(marks[marks.length - 1]);    // 75
        System.out.println(failedStudentMarks[0]);      // 0 (default value)

        marks[0] = 100;                                 // change 90 to 100

        for (int i = 0; i < marks.length; i++) {
            System.out.println("Value at index " + i + " is " + marks[i]);
        }

        for (int m : marks) {
            System.out.println(m);
        }
    }
}
```

Output

```
90
75
0
Value at index 0 is 100
Value at index 1 is 80
Value at index 2 is 10
Value at index 3 is 20
Value at index 4 is 50
Value at index 5 is 75
100
80
10
20
50
75
```

Compile with **`javac ArrayExample.java`** and run with **`java ArrayExample`**.

Common mistakes

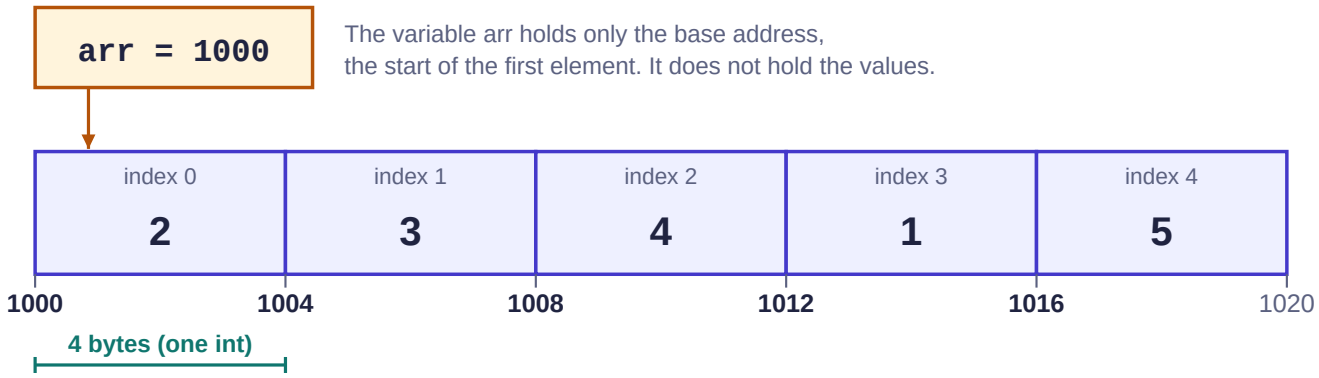
- Forgetting `[]` in the declaration, which makes a normal variable.
- Counting from 1. The first element is at index 0.
- Looping with `i <= marks.length`. It goes one step past the end, and Java stops the program with an `ArrayIndexOutOfBoundsException`.
- Writing `marks.length()` with brackets.
- Trying to make an existing array bigger. Its size is fixed.

5. How an array works inside memory

Section 4 is what you write. Now let us see what the computer does. This part is a favourite in interviews and in exams like GATE.

Take `int[] arr = {2, 3, 4, 1, 5};` and follow what happens:

1. The computer finds one continuous free chunk of RAM, big enough for all 5 elements.
2. An int takes **4 bytes** in Java, so each element gets a block of 4 bytes. In total, 5 × 4 = 20 bytes.
3. The blocks are placed side by side: the 2, then the 3 right next to it, then 4, 1 and 5. The row can sit anywhere in memory, but it always stays together.
4. The first block starts at some address. This is the **base address**. We assume 1000 here. The operating system picks the real number.
5. The variable `arr` does not contain the five values. It contains only the base address.



Key idea

Base address is the starting address (the lower boundary) of the first element. The array variable stores it, so every element can be found from there. When you use the array, you are really using the base address.

The address formula

Address of element at index i = Base address + (i × size of one block)

The block size depends on the type. In Java an int takes 4 bytes and a double takes 8 bytes. In some other languages the size of int is different, so always check the language you are using.

With base address 1000 and 4-byte blocks:

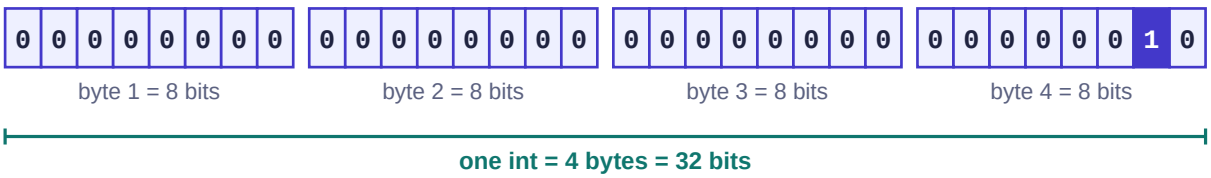
Element	Calculation	Address
index 0	1000 + 0 × 4	1000
index 1	1000 + 1 × 4	1004
index 2	1000 + 2 × 4	1008
index 3	1000 + 3 × 4	1012
index 4	1000 + 4 × 4	1016

Each address is the lower boundary of that block. The block of index 0 covers bytes 1000, 1001, 1002 and 1003, and the next block starts at 1004.

> **Good to know**
> You never add 4 again and again by hand. For an array of 1,00,000 elements you simply use the formula. When you write `arr[i]`, Java does exactly this calculation inside, then returns the value. That is why you never see addresses in a normal program.

Bytes and bits

- 1 byte = 8 bits. An int takes 4 bytes, which is 4 × 8 = **32 bits**.
- A bit is a tiny box that can hold only 0 or 1.
- A number is first converted to binary and then stored in those bits. This is how the number 2 sits inside one int block:



How negative numbers are stored (two's complement) and what most and least significant bits are is covered in digital design subjects. For arrays, it is enough to know that one int block is 32 bits. Hardware, the operating system and the language work together, and the language hides these details from you.

6. Why contiguous memory helps

A common interview question: what is the benefit of contiguous memory in an array? There are three.

1. Fast access by index

Elements are side by side and the index is known, so Java jumps straight to the element with the address formula. Reading `arr[99]` in a 100-element array needs no loop. The time stays constant however big the array is. This is written **O(1)**, the best time complexity there is.

	Array (contiguous)	Linked list (not contiguous, covered later)
Memory layout	Elements side by side	Elements scattered around memory
Reach element number 99	Calculate its address and jump there: O(1)	No formula possible. Start at the beginning and hop from one element to the next with a loop

2. Friendly to the CPU cache

When you read one element, the processor guesses you will soon want the data around it, and copies the neighbours into the fast cache memory. In an array the neighbours are the next elements of the same array, so the guess is right and the following reads come straight from the cache. With scattered data the neighbours are unrelated, so the cache does not help and the processor must go back to main memory.

3. Easier to manage

All the data is in one place, so memory is simpler to manage.

Non-contiguous structures have benefits of their own. You will see them later in the series.

7. Quick revision

Topic	Remember
What it is	Many values of one type in one variable, stored side by side
Size	Fixed when the array is created
Indexes	0 to size - 1
Declare	`int[] a;`
Create with a size	`a = new int[n];` (slots get default values: 0, 0.0, false, null)
Create with values	`int[] a = {90, 80, 10};` (declare and initialize in one statement)
Read, change	`a[i]`, `a[i] = value;`
Size of the array	`a.length` (property, no brackets)
Traverse	`for (int i = 0; i < a.length; i++)` or `for (int x : a)`
Array variable holds	The base address, not the values
Address of index i	Base + i × block size
Access time	O(1), and cache friendly

8. Questions to test yourself

Question	Answer
Can the size of an array change after it is created?	No. Only the values can change.

Question	Answer
Can one array hold different types?	A normal Java array holds one type. Mixed types are possible only with non-type-safe collections, a topic for later.
What does the array variable store?	The base address of the array.
How do you find the address of element i?	Base address + i × size of one block.
Why is `arr[i]` O(1)?	The address is calculated directly from the index, so no looping is needed.
What are the first and last valid indexes?	0 and size - 1.
What are the default values of int, boolean and String arrays?	0, false and null.
Is it length or length()?	For arrays, `length` is a property, so no brackets.
Benefit of contiguous memory?	Fast index access, cache friendly, easier to manage.

Practice calculations (added for revision, not from the lecture)

Question	Answer
Base address 500, int array. Address of index 6?	500 + 6 × 4 = **524**
Base address 2000, double array (8 bytes each). Address of index 3?	2000 + 3 × 8 = **2024**
An array has 100 elements. First and last index?	**0** and **99**

9. What comes next

The next video covers more discussion on arrays, how to solve a few array questions, and how to build an **Array ADT** (abstract data type) with the basic operations, then use it. Practice questions follow, and 2D arrays will be explained if viewers ask for them. The lecturer's own notes and code are shared in the GitHub repository linked in the video description.